Solver-Checked Reasoning with KV-Cache Invalidation

Results summary, rounds 1–3 · UW undergraduate research · October 2026

Question. A language model reasons step by step, and an SMT solver (Z3) checks each step against the problem's premises. We ask two things:

- Does step checking improve accuracy?
- When a step is refuted, can it be removed cheaply? Instead of truncating everything after it, we mask the refuted step and its dependents in the KV cache and keep the independent work.

Short answer.

- Checking does not improve accuracy at any model size we tried.
- The systems side works: checking asynchronously is 12–67% faster than waiting for each verdict.
- At 14B and 30B, masking dependents costs no accuracy and uses significantly fewer tokens than truncation.

1. Method

Task and models.

- **Task:** ProverQA-Hard, first-order-logic problems with answers True / False / Uncertain; Z3 reproduces the gold label on 498/500.
- **Models:** Qwen3-8B (n = 30, laptop GPU); Qwen3-14B and Qwen3-30B-A3B (n = 100 each, Colab L4). All are Q4_K_M quantized, run via llama.cpp.

Step format. Each step has the form `Step k [uses P2, S1]: <fact>`, followed by `FOL: <formula>`. The solver labels the formula *entailed*, *refuted* or *unknown* given the premises. Only refuted steps are rejected.

Conditions.

- **base:** ProverQA's own chain-of-thought prompt.
- **fmt:** the step format, unchecked.
- **sync:** wait for every verdict.
- **trunc:** asynchronous checking; on a refutation, cut the cache back to the refuted step and regenerate.
- **mask:** asynchronous checking; remove the refuted step and every step whose `[uses]` chain reaches it, and keep the rest.
- **refresh:** mask, then recompute the kept steps so their cache entries never saw the refuted one.

Masking deletes a middle range of the cache (`kv_cache_seq_rm`). That is equivalent to setting those positions' attention to $-\infty$ for all future tokens.

Latency. Checker latency $L \in \{1, 3, 8\}$ s is simulated on a token clock: $L = 1$ s is about 14 generated tokens. Modeled time per problem is 8.6 s + 72.4 ms × tokens + checker waits, which makes it comparable across machines.

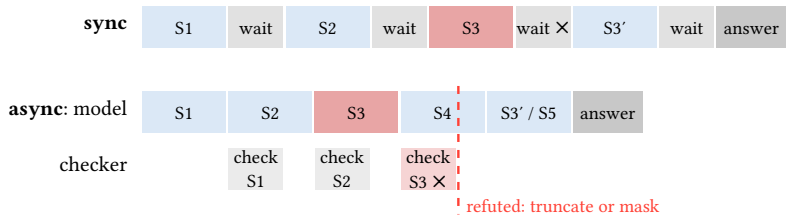


Figure 1: Blocking vs. asynchronous checking. With sync, every step waits L for its verdict. Async keeps generating while the solver works and acts on a refutation when it arrives (here, during S4).

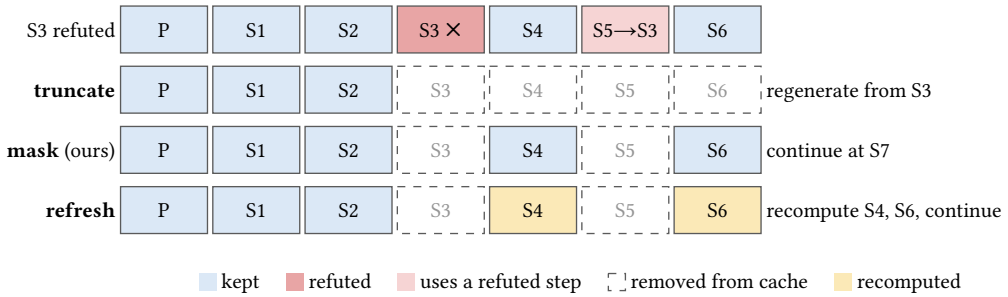


Figure 2: What each policy does to the KV cache when step S3 is refuted. Step S5 declared `[uses S3]`, so masking removes it too, while independent steps S4 and S6 stay. Truncation discards all of them.

2. Results

Condition	8B (n=30)	14B (n=100)	30B (n=100)	30B, no cap
base (plain CoT)	46.7	58	57	57
fmt (steps, unchecked)	56.7	66	67	71
sync	53.3	65	73	71
trunc, L = 1 / 3 / 8 s	63.3 / 63.3 / 63.3	70 / 70 / 70	72 / 73 / 73	71 / 71 / 71
mask, L = 1 / 3 / 8 s	60.0 / 53.3 / **36.7**	70 / 71 / 71	74 / 72 / 71	76 / 72 / 73
refresh, L = 8 s	53.3	73	71	72

Table 1: Accuracy (% correct). Always answering "Uncertain" scores about 40%. The 8B mask loss at L = 8 s is the only significant checked-vs-checked difference (0 wins / 8 losses vs. trunc, p = 0.008).

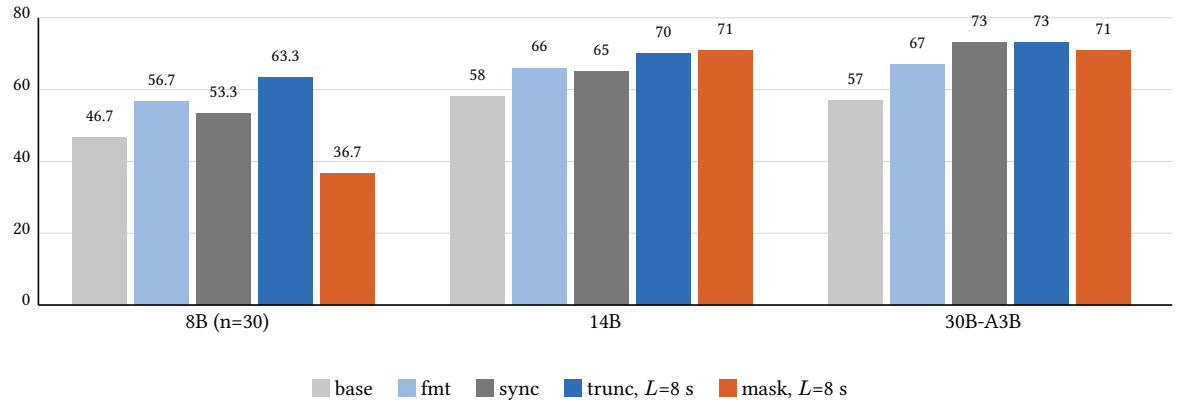


Figure 3: The step format helps (+8 to +14 points over CoT); the checker on top of it does not. Masking collapses at 8B with a slow checker but is fine at 14B and 30B.

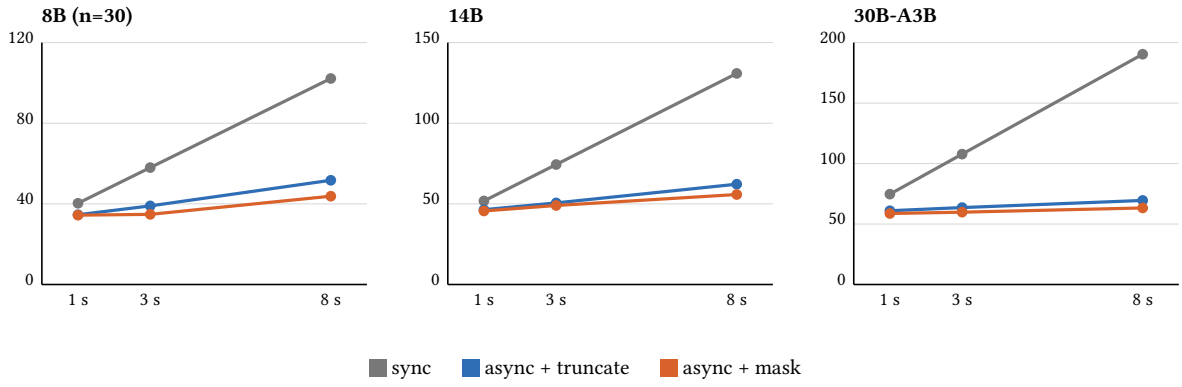


Figure 4: Modeled seconds per problem vs. checker latency. Blocking cost grows with every check; async cost barely moves.

Model	L	sync	trunc	mask	mask vs sync	mask vs trunc	fewer tokens, W/L (p)
8B (n=30)	1 s	40.3	34.6	34.4	−15%	−1%	4/3 (1.0)
	3 s	58.0	39.0	34.8	−40%	−11%	7/2 (0.18)
	8 s	102.2	51.7	43.8	−57%	−15%	9/3 (0.15)
14B	1 s	51.8	46.4	45.6	−12%	−2%	10/4 (0.18)
	3 s	74.4	50.6	49.0	−34%	−3%	16/9 (0.23)
	8 s	130.9	62.2	55.8	−57%	−10%	**32/10 (0.001)**
30B-A3B	1 s	74.7	61.0	58.7	−21%	−4%	**21/9 (0.04)**
	3 s	107.8	63.6	59.8	−45%	−6%	**24/9 (0.01)**
	8 s	190.4	69.5	63.3	−67%	−9%	**26/10 (0.01)**

Table 2: Modeled seconds per problem. Last column: problems where mask generated fewer / more tokens than trunc (exact sign test; bold = p < 0.05). Mask is 1–15% faster than trunc, the gap growing with latency.

3. Findings

1. **The checker does not improve accuracy at any scale.** Checked (sync) minus unchecked (fmt) is −3 points at 8B, −1 at 14B and +6 at 30B ($p = 0.11$). The step format itself is what helps: +8 to +14 points over plain CoT, significant at $p = 0.04$ for the 30B in the no-cap run.
2. **Asynchronous checking is the robust systems result.** It is 12–67% faster than blocking on every model, with the gap growing with checker latency (Figure 4), and costs no accuracy.
3. **Masking depends on model scale.**
   - At 8B, masking with a slow checker lost 8 problems to truncation and won none. By the time a verdict arrived, later "independent" steps had already absorbed the refuted step's content without declaring it in `[uses]`. Recomputing (refresh) recovered 5 of the 8.
   - At 14B and 30B, masking costs no accuracy (all $p \geq 0.18$), and refresh adds nothing.
   - At 30B, masking uses significantly fewer tokens than truncation at every latency, saving 4–9% of time.
4. **Ablation: removing the step cap (20 → 60 steps) changed no accuracy result** (every condition $p \geq 0.39$). It exposed repetition loops: 20–29% of problems repeat one formula until the new cap, in unchecked runs too. Mean tokens nearly double (617 → 1179 for fmt) while the median barely moves (720 → 767). The 20-step cap is a reasonable stopping rule.
5. **Context use.** Peak live KV cache is about 2,200–2,500 tokens per problem. Masked spans are small (19–38 tokens at peak), so masking saves by regenerating less after a refutation, not by keeping a smaller cache.

4. Limitations and next steps

Limitations.

- Checker latency is simulated rather than coming from a real slow checker.
- Masking relies on the model's self-declared dependencies, which the 8B violated.
- The results come from one benchmark.
- The 8B runs use n = 30.

Next steps.

1. Use a real slow checker, such as an LLM autoformalizer sharing the GPU, or Lean.
2. Make masking exact by letting each step attend only to the prompt and its declared parents (block-sparse attention). Undeclared reliance then becomes impossible by construction.